

Teoria-AAED-buena-de-verdad.pdf



MDMA



Análisis de Algoritmos y Estructuras de Datos



2º Grado en Ingeniería Informática



**Escuela Superior de Ingeniería
Universidad de Cádiz**

TEORIA AAED

1.PREGUNTAS GENERALES

- **¿Qué es un TAD?**

Un TAD es un conjunto de valores y operaciones que se definen mediante una especificación independiente de cualquier representación. Cumple con los principios de abstracción, ocultación de la información.

- **¿Qué importancia tiene la especificación en la creación y utilización de un TAD?**

Al crear un TAD el usuario debe ser capaz de utilizarlo sin conocer los detalles internos de su construcción. Así pues la especificación es una descripción del TAD, del dominio y de sus operaciones que está a disposición del usuario para que pueda usarlo. Está formado por la especificación semántica y la sintáctica.

- **¿Qué es la especificación sintáctica?**

Indica cómo usar las operaciones, es decir, la forma correcta de escribir la llamada a las operaciones del TAD. También indica el tipo de operandos y resultado.

- **¿Qué es la especificación semántica?**

El significado de las operaciones. Informa de que realizará cada operación del TAD y las condiciones que deben cumplirse antes de la ejecución (precondición) y las que se cumplirán después (postcondición).

- **¿Qué es la implementación de un TAD?**

La parte del TAD conocida solamente por el diseñador del TAD y es la definición de la estructura de datos que soporta el TAD y la codificación de las operaciones de forma que muestre los comportamiento y las propiedades especificadas.

- **¿Cómo se realiza la ocultación en C?**

Mediante la abstracción de las implementaciones de las operaciones.

Debería ser también la ocultación de los tipos de datos, pero esto en C no es posible ya que no se puede ocultar el archivo de cabecera ".h".

Las implementaciones de las operaciones se introducen en un archivo ".c" oculto, es decir un archivo ".o".

- **Principio fundamental de los TAD: independencia de la representación.**

El principio de la independencia de la representación dicta que no importa como esté implementado el TAD mientras se cumpla la especificación del mismo, lo que implica que un mismo TAD se puede representar mediante diferentes implementaciones.

- **¿Existe alguna relación entre la ocultación de información y la independencia de la representación?**

Con la independencia de la representación se consigue que el usuario solo necesite conocer la especificación del TAD sin necesidad de saber cómo está representado. La ocultación de información se apoya en la independencia de la representación y viceversa.

- **Ante la posibilidad de que no se verifiquen las precondiciones de una operación de un TAD, ¿Qué decisiones de diseño puede implementar la operación?**

Controlar los posibles fallos, lo ideal sería que notificase un error advirtiendo al usuario. Si el error es muy grave, detener el programa, ya que puede dar un resultado indeterminado o erróneo que después se reflejará en el mal funcionamiento.

- **¿Qué sucede si después de la ejecución de una determinada operación, no se cumplen las precondiciones de la misma?**

No ocurriría nada ya que no se cumple la precondición, pero no importa porque la precondición debe cumplirse antes de la ejecución de la operación, no después.

- **¿Qué ventajas tiene realizar la especificación de un TAD después de hacer la implementación?**
No solo no tiene ventajas, sino que además no es posible. Porque el principio de independencia nos dice que independientemente de la representación que se use, la especificación del TAD es siempre la misma.
- **Concepto de dualidad.**
Permite "traspasar" las propiedades de O a Ω y viceversa.
 $f - O(g) \Leftrightarrow g - \Omega(f)$
 $O(f) = O(g) \Leftrightarrow \Omega(f) = \Omega(g)$
- **Ordene las siguientes funciones según $< o$:**

1 normal y log
2 raíces y divisiones
3 n y $n \log$
4 n^2
5 exponenciales
6 factorial
7 exponencial con exponente $3n$

2. PILAS

- **¿Qué es una pila?**
Secuencia de elementos de un tipo determinado en la cual se pueden añadir y eliminar elementos sólo por uno de sus extremos, llamado tope. En una pila el último elemento añadido es el primero en salir de ella, por lo que es una estructura LIFO.
- **¿Cuándo utilizaremos una representación estática del TAD pila? Ventajas e inconvenientes.**
Cuando sepamos el número máximo de elementos que vamos a almacenar en ella y todas las pilas que vayamos a usar sean del mismo tamaño.
Ventajas: Ocupa menos espacio que la enlazada ya que no requiere punteros.
Inconvenientes: exige fijar el tamaño máximo de la pila en tiempo de compilación por lo que el tamaño tendría que ser constante y el mismo para todas las pilas que creamos. Se puede desperdiciar memoria y se puede llenar.
- **¿Cuándo utilizaremos una representación pseudoestática del TAD pila? Ventajas e inconvenientes.**
Se usa cuando sea conveniente emplear pilas de distinto tamaño.
Ventajas: Nos permite fijar distintos tamaños en tiempo de ejecución (este tamaño será fijo durante todo el tiempo de la vida de la pila), por lo que un tamaño inicial no determina el tamaño de todas. Ocupa menos espacio que la implementación de celdas enlazadas porque no requiere un puntero por elemento.
Inconvenientes: Tamaño constante durante la vida de la pila. Se puede llenar o desaprovechar memoria si es demasiado grande.
- **¿Cuándo utilizaremos una representación dinámica del TAD pila? Ventajas e inconvenientes.**
Cuando no necesitemos controlar o no sepamos el número máximo de elementos que va a tener una pila.
Ventajas: El tamaño de las pilas es variable, siempre tendrá el tamaño necesario para almacenar los elementos que contenga la pila en cada instante.
Inconvenientes: Consumo elevado de memoria debido al uso de un puntero por cada elemento.
- **¿Por qué en la representación dinámica del TAD pila se representa la pila como un puntero a las estructuras de datos diseñadas?**
La razón de introducir un puntero es hacer que el paso de parámetros a las operaciones del TAD sea por referencia, de modo que los cambios en los parámetros formales se vean reflejados en los parámetros reales. En algunas operaciones (pop o push) es necesario modificar el puntero haciendo que apunte al nuevo tope. Por tanto ese puntero hay que pasarlo por referencia.
- **¿Por qué en la representación estática y pseudoestática de un TAD pila, el registro que representa la pila se pasa por referencia sin ser necesario? (Puesto que el registro que representa a la pila no se modifica al añadir o quitar elementos de ella)**
No es necesario pasarlo por referencia. Sin embargo, en virtud del principio de independencia

de la representación, los parámetros de tipo pila tienen que ser transferidos por referencia en las diferentes implementaciones.

- **¿Tiene algún sentido representar una pila con una lista doblemente enlazada?**
No, ya que en una lista doblemente enlazada se usan 2 punteros (anterior y siguiente) y en las pilas solo se inserta y elimina por el tope, como no hay que acceder al elemento anterior a otro no tiene sentido usar esa representación.
- **¿Cuándo utilizaremos la representación circular del TAD pila?**
Esta representación no existe para el TAD pila.
Tal y como se define el TAD pila, una pila es una secuencia de elementos de un tipo determinado, en la cual se pueden añadir y quitar elementos solo por un extremo (tope), por lo tanto carecería de sentido usar un puntero que se utilizara para apuntar a la base de la pila.
- **¿Cómo se podría eliminar un elemento cualquiera de una cola/pila?**
No se podría hacer, ya que tanto para pilas como para colas tenemos una especificación que ha de cumplirse, y en ninguna de las 2, se dice como hacerlo. Una forma de hacerlo es ayudarse de una pila/cola auxiliar en la cual volcamos la que tenemos.
- **¿En qué programas utilizarías el TAD Pila?**
En los casos en los que el usuario desee extraer elementos en el orden inverso en el que se insertan. También si desea que las operaciones sean O constante. Ej: líneas de texto.

3. COLAS

- **¿Qué es una cola?**
Una cola es una secuencia de elementos en la que las operaciones se realizan por los extremos: Las eliminaciones se realizan por el extremo llamado inicio o frente de la cola, las inserciones son añadidos por el otro extremo, llamado fondo o fin de la cola.
En una cola el primer elemento añadido es el primero en salir de ella, por lo que es una estructura FIFO.
- **Ventajas e inconvenientes de una representación vectorial del TAD cola.**
Ventajas: Ocupa un espacio fijo de memoria, el cual está representado por una estructura que contienen un vector que almacena todos los elementos, un entero que almacena el tamaño máximo del vector y otro entero que almacena el fin de la cola.
Inconvenientes: La operación `Pop()`, es muy costosa en función del nº de elementos.
- **Ventajas e inconvenientes de una representación circular del TAD cola.**
Ventajas: esta representación soluciona el inconveniente de la representación vectorial. Y además, como el principio y el fin de la cola son consecutivos el orden de las operaciones de eliminación e inserción se reducen.
Inconvenientes: Distinguir entre una cola llena o vacía, ya que la posición de los índices inicio y fin sería la misma en ambos casos. Las soluciones propuestas serían añadir un indicador de si está o no vacía la cola o añadir una posición más al vector, la cual siempre estaría vacía. Si entre inicio y fin hay una posición vacía, la cola está llena.
- **Ventajas e inconvenientes de la representación con celdas enlazadas del TAD cola.**
Ventaja: Acceder a fin con coste 1 y a inicio con coste 2.
- **Ventajas e inconvenientes de la representación con celdas doblemente enlazadas del TAD cola.**
Ventajas: los accesos a ambos extremos de la cola se consiguen en coste $O(1)$.
Inconvenientes: requiere dos punteros.
- **Eres un diseñador del TAD Cola y un usuario te solicita una operación de acceso al n-ésimo elemento de la misma. ¿Incluirás esta operación en el TAD?**
No, ya que esta operación no es una operación del TAD Cola. Se tendría que crear un TAD nuevo correspondiente a las necesidades del usuario.

- **¿Por qué en la implementación vectorial circular de las colas, existe una posición que siempre está vacía?**
Existe una posición que siempre deba estar vacía para que se pueda distinguir cuando la cola está vacía o llena. Estará vacía cuando inicio sigue a fin, y llena cuando entre ambas posiciones quede una libre.
- **La representación de una cola mediante estructuras enlazadas, ¿Por qué se representa usando dos punteros a cada extremo de la misma?**
La representación de una cola mediante estructuras enlazadas podría realizarse usando un solo puntero a uno de los extremos de la misma, o mediante dos punteros, uno a cada extremo, como en este caso.
Una posible representación, utilizando tan sólo un puntero, sería apuntar al fin de la misma y enlazar el último elemento con el primero, de forma que se accedería al fin de la cola en coste $O(1)$ y al inicio en coste $O(2)$.
Otra posibilidad, acorde con los accesos al TAD cola, es representarla mediante dos punteros que apunten al inicio y fin de la cola. Su ventaja es que el acceso a ambos extremos de la cola se consigue en coste $O(1)$. Su inconveniente es que requiere dos punteros en vez de uno.
- **¿Para qué se utiliza el nodo cabecera en el TAD Cola, en su implementación de celdas enlazadas?**
- **¿Con la implementación de colas mediante un vector circular de tamaño n cuantos elementos pueden almacenarse?**
Se podrían almacenar n-1 elementos, ya que una de las posiciones del vector se debe reservar para poder distinguir cuando está llena o vacía.
- **Con la representación de colas mediante una estructura enlazada, con puntero al final y circular ¿Cuántos elementos pueden almacenarse?**
Pueden almacenarse todos los elementos que se quiera, ya que la representación de colas mediante una estructura enlazada es dinámica y por tanto no hay un límite de elementos a almacenar.
- **Define una estructura de datos basada en el vector circular para almacenar una bicola de valores de tipo T y escribe el código de las operaciones: void BicolaPopFin(Bicola B) y void BicolaPushInicio(T x, Bicola B).**

```

class bicola {
public:
    bicola(size_t tmax);
    void BicolaPopFin();
    void BicolaPushInicio(const T& x);
private:
    T *elemento;
    int tmax, lmax;
    int inicio, fin;
}

Bicola(size_t tmax){
    elemento(new T[tmax+1]);
    lmax = tmax+1;
    inicio = 0;
    fin = tmax;
}

```

```

void BicolaPopFin(){
    assert(!vacía());
    fin = fin-1 % Lmax;
}

void BicolaPushInicio(const T& x){
    assert(!llena());
    inicio = inicio-1 % Lmax;
    elemento[inicio] = x
}

```

- **Comente la siguiente afirmación: En la representación del TAD cola circular mediante una estructura de celdas enlazadas es necesario usar un nodo cabecera.**
El TAD cola circular no existe. Existe el TAD cola con una representación circular.

4. LISTAS

- **¿Qué es una lista?**
Una lista es una secuencia de elementos de un tipo determinado. Se representa de la forma $L = (a_1, a_2, \dots, a_n)$, donde $n \geq 0$ y longitud = n . Si $n = 0$ es una lista vacía.
Posición: Lugar que ocupa un elemento en la lista. Los elementos están ordenados de forma lineal según las posiciones que ocupan. Todos los elementos salvo el primero tienen un único predecesor, y todos menos el último tienen un único sucesor.
Fin: posición siguiente a la del último. Esta posición no está ocupada nunca por ningún elemento.
- **¿El TAD lista y el TAD lista circular son TADs diferentes?**
Sí, son dos TADs diferentes porque tienen operaciones distintas.
- **¿En una representación vectorial de una lista, como representamos la posición fin?**
Mediante la longitud del vector.
- **Ventajas e inconvenientes de la implementación vectorial del TAD Lista.**
Ventajas: Las posiciones podrían representarse con enteros y se puede permitir el acceso directo a un elemento por dicho entero.
Inconvenientes: Las operaciones insertar y eliminar pueden ser muy costosas ya que se habrán de mover el resto de elementos. Hay que especificar el tamaño máximo de la lista.
- **Ventajas e inconveniente de la implementación dinámica del TAD Lista.**
Ventajas: Usa solo el espacio necesario para almacenar los elementos que tiene la lista en cada instante.
Inconveniente: si la lista es demasiado pequeña, puede ser que se "desaproveche" más espacio para albergar al puntero de cada nodo.
- **¿Por qué surge el nodo cabecera en el TAD Lista?**
Porque si no existiese el nodo cabecera, nos estaríamos saltando el principio de la independencia de la representación. Usando el nodo cabecera estaremos cumpliendo la especificación del TAD en las funciones insertar y eliminar para que todos los elementos tengan predecesor y así ninguno tenga que tratarse de forma diferente.
Con el nodo cabecera se consigue reducir el O de insertar y eliminar de $O(n)$ a $O(1)$, ya que con la cabecera no es necesario recorrer la lista en busca de la posición anterior a la que se desea insertar o eliminar. Buscar anterior y fin son $O(n)$. Las demás $O(1)$.
- **¿Con que problema fundamental nos encontramos si suprimimos L en los argumentos de las operaciones del TAD lista?**
Nos estamos saltando el principio de independencia de la representación del TAD, en las operaciones: Insertar(), Eliminar(), Anterior() y Siguiente().

Si suprimimos L en los argumentos, las especificación y utilización del TAD dejan de ser independientes de la implementación.

- **¿Cuándo se utiliza el TAD Lista Circular?**
Cuando no hay definido un primer y un último elemento de la lista, es decir, cuando no se sabe cuales son los extremos de la lista, por lo que todos los nodos están seguidos unos de otros. Es innecesario el nodo cabecera y la operación fin().
- **¿Diferencia entre TAD Lista Circular y TAD Lista?**
La diferencia principal es que, en la lista circular todos los elementos tienen sucesor y predecesor. No hay posición primera ni fin, solo una llamada a inipos() que devuelve una posición cualquiera para tomarla como referencia. Tampoco hay nodo cabecera.
- **¿Para que se utiliza una implementación mediante estructura enlazada doble?**
Se utiliza para reducir el orden de las operaciones anterior y fin de $O(n)$ a $O(1)$.
- **¿Cuándo es conveniente utilizar una implementación mediante estructura enlazada doble en el TAD Lista?**
Cuando sea necesario recorrer eficientemente una lista, tanto hacia adelante como hacia atrás o cuando se quiera que la operación anterior sea rápida ($O(1)$).
- **En la lista Circular, es posible implementar alguna operación de orden logarítmico?**
No, todas las operaciones son de orden 1 o n, ya que o se hace referencia al elemento al cual apunta el puntero o a alguno contiguo. Para las operaciones de orden n se ha de recorrer toda la lista hasta hallar el elemento que se desea.
- **¿Qué condición tiene que cumplir una lista para que la búsqueda sea logarítmica?**
Para que fuera logarítmica debería ser una búsqueda dicotómica, la cual solo se usa el vectores ordenados y no en Listas.
- **¿Por qué en la especificación del TAD lista, en la operación anterior, las precondiciones son $L=(a_1, a_2, \dots, a_n)$ $2 \leq p \leq n+1$?**
El hecho de que p deba ser mayor o igual que 2 y menor o igual que n+1 viene dado porque la primer posición no tiene anterior, y la posición fin, aunque no tenga elementos, si posee una posición anterior.
- **¿Por que en la especificación del TAD lista, en la operación siguiente, las precondiciones son $L=(a_1, a_2, \dots, a_n)$ $1 \leq p \leq n$?**
El hecho de que p deba ser mayor o igual que 1 y menor o igual que n viene dado porque el último elemento no tiene siguiente (el siguiente es la posición fin).
- **¿Por qué en la especificación del TAD List Circular, en las operaciones anterior y siguiente, las precondiciones son $L=(a_1, a_2, \dots, a_n, a_1)$ $1 \leq p \leq n$?**
Porque todas y cada una de las posiciones de la lista circular poseen una posición anterior y una posición siguiente.
- **¿Qué pasaría si insertaras o eliminaras en la primera posición en una implementación de lista mediante estructura enlazada sin cabecera?**
Habría que tratarlos de forma especial porque el primer nodo no tiene ninguno que lo preceda. Además estaría incumpliendo la independencia de la representación.
- **Representando el TAD lista mediante un vector. ¿Es posible evitar el coste $O(n)$ para inserciones y borrados en los extremos?**
Si, implementándola mediante un vector circular.
- **¿Por qué se pasa el parámetro posición por referencia en una lista en las operaciones insertar y eliminar en la implementación mediante estructuras enlazadas?**
Al modificar el 1er elemento será necesario cambiar ese puntero inicial para que apunte al nuevo primer elemento, así que ese puntero hay que pasarlo por referencia.

